

Java-Based Offline Voice Assistant

Project Report

Submitted by: Jai Dixit

Program: B.Tech CSE (AI & ML)

Institution: VIT Bhopal University

Submission Platform: VITyarthi

1. Introduction

This report documents a Java-based, mostly-offline voice assistant built as a self-directed project. The assistant listens to the user's voice through the computer microphone, converts the speech into text using an offline model, matches the text against a set of known commands, executes the corresponding action on the machine, and speaks a confirmation back to the user. The project applies core Java concepts including multithreading, the Command design pattern, exception handling, and the Java Sound API.

2. Problem Statement

Everyday desktop tasks such as checking the time, opening an application, or searching the web require switching context to the keyboard and mouse, even for very small actions. This project addresses that friction by allowing these small, frequent tasks to be triggered purely by voice, without needing constant internet access, since the speech recognition itself runs entirely offline.

3. Functional Requirements

- System controls: report the current time and date; lock or shut down the computer.
- App launching: open calculator, notepad, terminal, or a code editor.
- Web browsing: search Google, open YouTube results, or look up Wikipedia articles.
- Media playback: play, pause, or skip local MP3 tracks.
- Timers and alarms: set a countdown timer that beeps and speaks when it finishes.
- Quick answers: fetch a one-line weather report, or tell a random joke.

Input/output structure: input is a spoken sentence, converted to text by the SpeechRecognizer. Output is (a) an OS-level side effect, such as an application opening or the machine locking, and (b) a spoken sentence confirming what happened, produced by the SpeechSynthesizer.

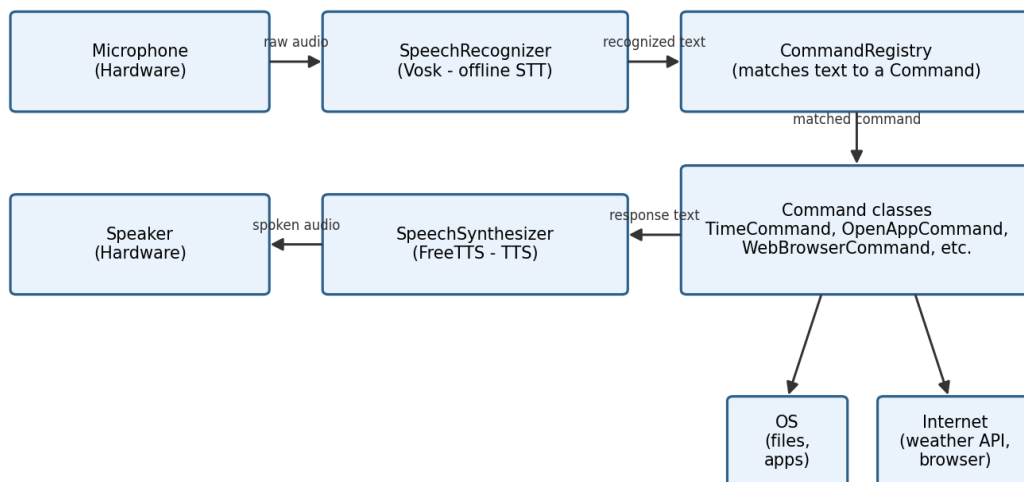
4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Recognition and response happen within 1-2 seconds; the mic and timers run on background threads so no
Reliability	Every risky operation (mic, file, network, process launch) is wrapped in try/catch so one failure cannot crash
Usability	Short, natural phrases are accepted; every action gets a spoken confirmation.
Resource efficiency	A small ~40MB offline model is loaded once at startup, keeping memory and startup time low.
Maintainability	New commands are added as a single new class implementing Command, with no changes to existing clas
Error handling strategy	Failures return a plain-English spoken message instead of crashing or failing silently.

5. System Architecture

The system is organized into three layers: an audio layer (microphone capture and speech synthesis), a command layer (the Command design pattern implementation), and an entry-point layer (Main, which wires the two together and manages the background listening thread).

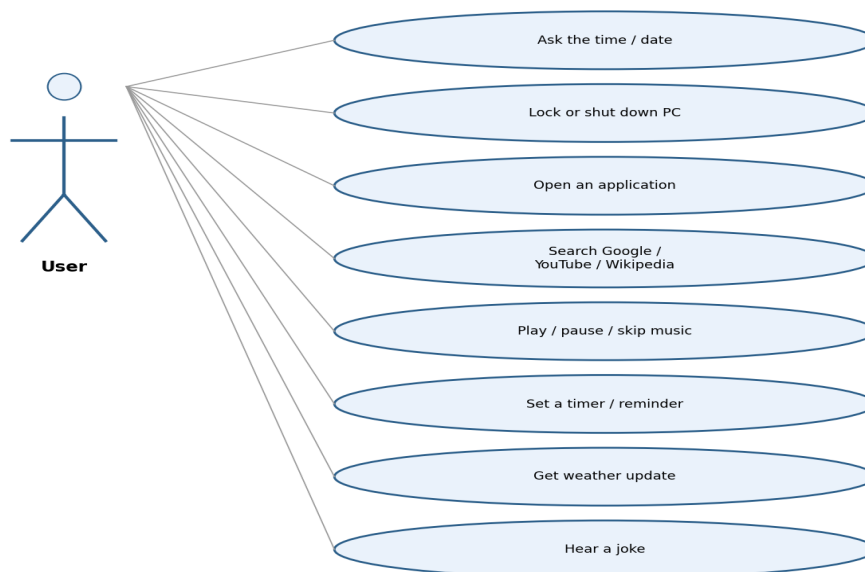
System Architecture Diagram



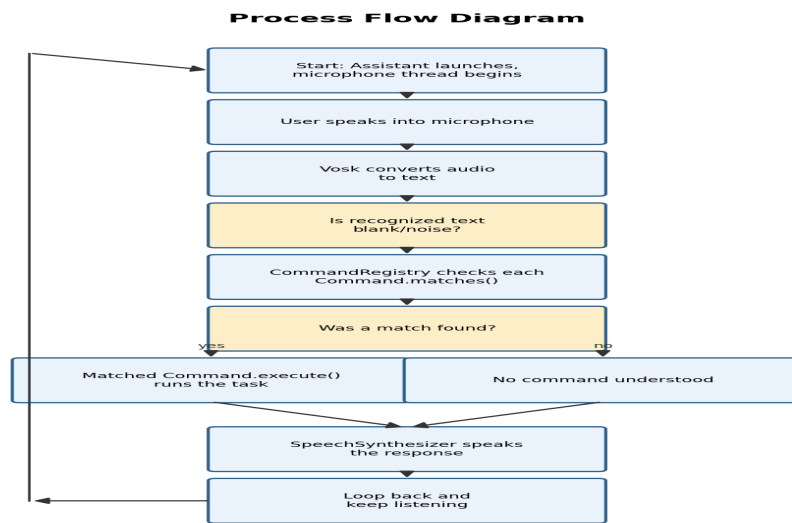
6. Design Diagrams

6.1 Use Case Diagram

Use Case Diagram

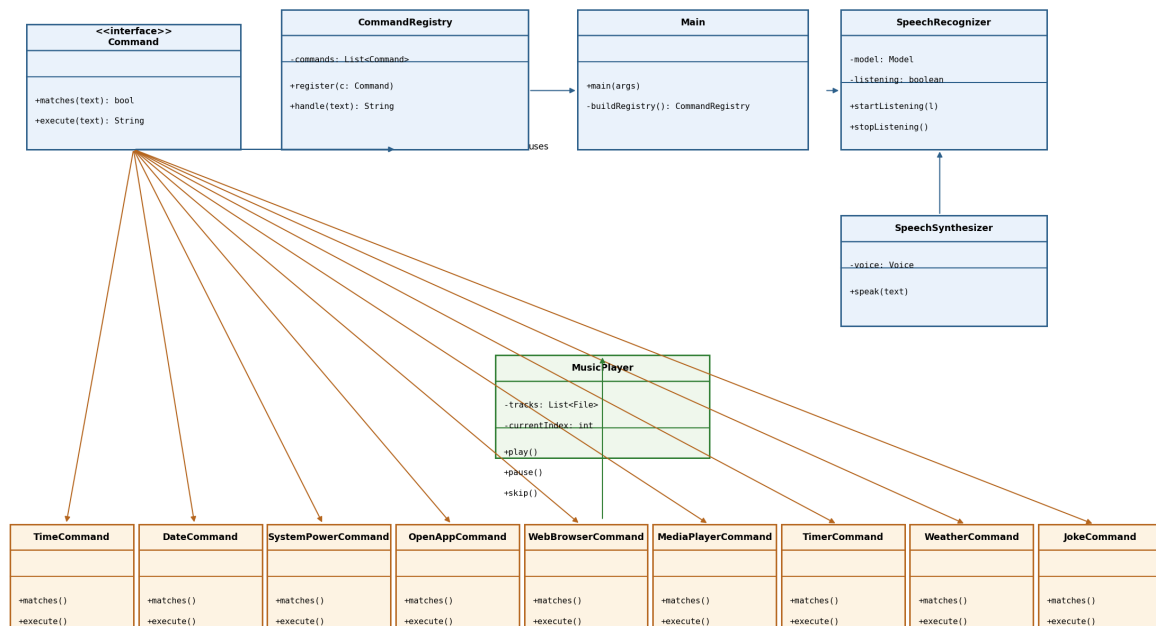


6.2 Process Flow / Workflow Diagram



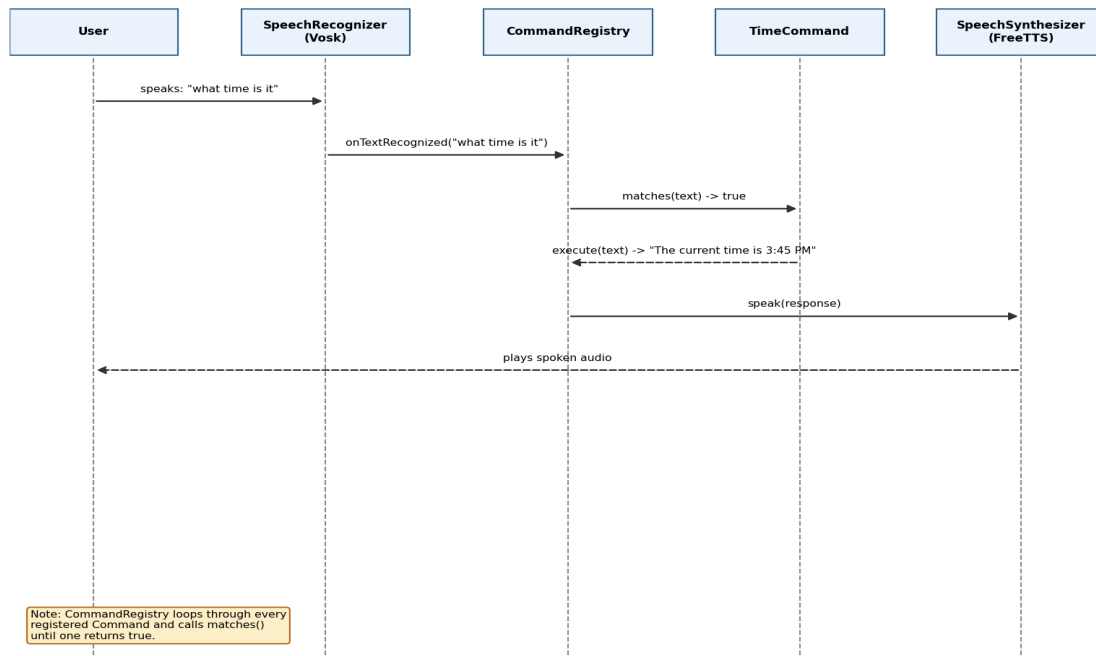
6.3 Class Diagram

Class Diagram



6.4 Sequence Diagram

Sequence Diagram - "What time is it" command



6.5 ER Diagram / Database Schema

Not applicable. The project does not use a database or persistent storage; all state exists only in memory while the program runs, and MP3 files are read directly from a folder on disk.

7. Design Decisions & Rationale

7.1 Why the Command Design Pattern

Each voice action is implemented as its own class behind a common Command interface, rather than one large if-else chain. This means a new command can be added by writing one new class and registering it, without touching or risking any existing command's logic — directly satisfying the maintainability non-functional requirement.

7.2 Why Vosk over Sphinx4

Both are offline speech-to-text options considered for this project. Vosk was chosen because it ships small pre-trained models (as low as ~40MB), has an actively maintained API, and integrates directly with raw PCM audio from the Java Sound API without extra configuration files. Sphinx4 is older, requires more manual acoustic/language model setup, and has a larger footprint for comparable accuracy.

7.3 Why the OS's built-in speech engine instead of a library like FreeTTS

An early version of this project used the FreeTTS library for text-to-speech. FreeTTS was dropped because its jars are hosted on an unreliable third-party repository that Maven frequently fails to resolve, which would force anyone cloning the project from GitHub to manually hunt down the jar. Calling the operating system's own built-in speech engine (Windows System.Speech via PowerShell, macOS 'say', Linux 'espeak') through ProcessBuilder removes that dependency entirely, so ``mvn clean package`` succeeds for any reviewer with zero extra setup on Windows or macOS.

7.4 Why keyword matching instead of true NLU

Given the project's scope and time constraints, commands are matched using keyword checks (`Command.matches()`) rather than a trained intent-classification model. This keeps the system simple, fast, and fully explainable, at the cost of requiring the user's phrasing to roughly match the coded keywords.

8. Implementation Details

The project is a standard Maven project with three main packages under `com.voiceassistant`: **audio** (SpeechRecognizer, SpeechSynthesizer), **command** (the Command interface, CommandRegistry, and nine concrete command classes), and **util** (MusicPlayer). `Main.java` is the entry point: it builds the CommandRegistry, starts the SpeechSynthesizer, then starts the microphone-listening loop on its own Thread so it never blocks the rest of the program. Recognized text is lower-cased and trimmed, then passed to `CommandRegistry.handle()`, which loops through registered commands calling `matches()` until one returns true, then calls that command's `execute()`.

Key Java concepts applied: multithreading (mic thread and each timer thread), the Command design pattern, I/O streams (microphone byte stream, MP3 file stream), ProcessBuilder and the Desktop API (launching apps and the browser), regular expressions (extracting timer duration from a sentence), and exception handling around every operation that can fail at runtime.

9. Screenshots / Results

[Insert screenshots here before submitting: (1) the project open in IntelliJ, (2) a terminal/console showing a recognized command and its spoken response text, (3) the mvn test output showing all tests passing.]

10. Testing Approach

Unit tests (JUnit 5) were written for the parts of the system that do not depend on hardware or the network, so they run quickly and deterministically:

- TimeCommandTest / DateCommandTest — verify matches() correctly identifies relevant sentences and rejects unrelated ones, and that execute() returns a correctly-shaped response.
- JokeCommandTest — verifies a non-blank joke is returned across many random draws.
- CommandRegistryTest — uses fake Command implementations to verify dispatch order, correct routing, and the fallback message when nothing matches.

Manual end-to-end testing was performed by running Main.java and speaking each supported command listed in the Functional Requirements section, confirming both the console log of recognized text and the spoken response.

11. Challenges Faced

- Offline recognition accuracy: the small Vosk model trades accuracy for size and speed, so accuracy is noticeably lower than a cloud speech API, especially with background noise.
- MP3 pause/resume: the JLayer library used for playback does not support a true pause — the current implementation stops playback rather than resuming from the same position.
- Cross-platform system commands: locking and shutting down the computer requires different native commands on Windows, macOS, and Linux, handled with OS detection in SystemPowerCommand.
- Keeping the assistant responsive: without running the microphone on a separate thread, the whole program would freeze while waiting for speech.

12. Learnings & Key Takeaways

- The Command design pattern makes a growing feature set much easier to manage than a single large conditional block.
- Offline models involve a real accuracy-vs-resource-usage tradeoff, which is a recurring theme in applied machine learning, not just an inconvenience specific to this project.
- Defensive exception handling around every hardware- and network-facing call is essential for a program meant to run continuously.

13. Future Enhancements

- Use a larger Vosk model or a cloud speech-to-text API for higher accuracy.
- Add fuzzy or synonym-based matching so more phrasings of the same command are understood.
- Add a simple graphical interface instead of console-only output.
- Persist a log of past commands and responses to disk for review.

14. References

- Vosk Speech Recognition Toolkit — <https://alphacephei.com/vosk/>
- Microsoft System.Speech (Windows built-in TTS, invoked via PowerShell) — <https://learn.microsoft.com/en-us/dotnet/api/system.speech.synthesis>
- eSpeak text-to-speech (Linux) — <http://espeak.sourceforge.net/>
- JLayer (javazoom) — <http://www.javazoom.net/javalayer/javalayer.html>
- Java Sound API documentation — <https://docs.oracle.com/javase/tutorial/sound/>
- wttr.in weather API — <https://wttr.in>
- Oracle Java Platform, Command design pattern reference material used for class structuring.